Thursday Workbook: Clean Up a Playlist, Test First

Week 11 · Pair Reasoning · work in pairs · all data is made up

On Tuesday you watched one process from start to finish. Today **you and a partner run the same process yourselves**, on a different problem:

> A playlist has the same song in it more than once. Some listings are spelled differently ("Slow Orbit", "Slow Orbit (Remastered)", "slow orbit"). Write a small program that finds the repeats and keeps one copy of each song.

You will not write all the code by hand. You may use Claude. **But you write the predictions, the counterexample, and the summaries yourselves,** and you run every test yourselves. That is where the learning is.

Words in **bold** are explained the first time they appear and again in the Glossary page. If a word is not explained, that is our mistake: circle it and tell us.

Part A. Before you touch anything

A1. Why write the test first? (two minutes)

You taste a soup *before* you serve it. You press the smoke-alarm button *before* there is a fire. In both cases you check that the thing does its job while it is cheap to find out that it does not.

A **test** is a tiny program that checks one thing about your other program and answers **pass** or **fail**. A **unit test** checks one small piece (usually one function). Writing the test *first* means you decide what "correct" looks like before you are tempted to bend the definition to fit what you built.

A2. Red and green

We run the tests with a tool called **pytest**. It prints one of two things:

- **Red**: at least one test failed. *This is normal and good.* Red means the test can actually notice a problem. A test that has never been red might not be able to fail at all.
- **Green**: every test passed.

Our rhythm for almost every step: **make it red, then make it green.** If a step says "expect red", a red result means you did it right.

A3. The same three beats at every step

Beat	What you do	Why
PREDICT	Write down what you think will happen, *before* running.	Guessing first is how you notice what you did not understand.
RUN	Run `python3 -m pytest -q`. Look at red or green.	The computer is the referee, not you and not Claude.
EXPLAIN	In your own words, say why it came out that way.	If you cannot explain it, you do not own it yet.

Being wrong in PREDICT is fine and useful. Cross out your guess, do not erase it.

A4. Set up (about 5 minutes; do it together)

You need Python 3.9 or newer and `pytest`.

1. Copy the `starter` folder to a new folder, for example `playlist-lab`. Open a terminal in it.
2. Install pytest if you need to: `python3 -m pip install pytest`
3. Check it works: `python3 -m pytest -q` You should see `no tests ran` (or "collected 0 items"). That is correct: there are no tests yet.

What is in the folder: `playlist/track.py` (what a **Track** is: an id, a title, an artist, a length in seconds), `tests/fixtures.py` (made-up sample tracks; called **fixtures** because they are fixed sample data the tests reuse), and `tests/relations.py` (a ready-made helper that hunts for counterexamples; you will meet it in Step 02).

Do not edit `tests/relations.py` or `playlist/track.py`. Everything else you may change.

Paper path. No laptop, or Python will not install? Do every PREDICT and EXPLAIN box on paper, and do the by-hand checks the steps mention (count the gaps, draw the arrows, fill in the property table). Steps 04 onward need a computer to see red and green, so a paper-path pair does Steps 01 to 03 and Step 07 on paper, plus the summary. You will still reach the main idea. The facilitator can give you a printed pytest screen to read.

A few Python words you will see (you do not need to write them):

- A **function** is a named recipe: `length_label(90)` runs the recipe on 90 and gives back an answer.
- A **stub** is a function that exists but does not do its real job yet, for example it just returns `None` ("nothing").
- An **assert** is one line in a test saying "I claim this is true"; if it is false the test fails.
- To **raise** an error such as `ValueError` means to stop and shout "this input is not allowed." A test can check that the shout happens.
- A **tuple** is a short fixed list written `(a, b, c)`.
- A **witness finder** (in `tests/relations.py`) is a function that hunts for a counterexample. It gives back the tuple it found, or `None` if it found none.
- **Import lines** in tests look like `from relations import transitive_failure` and `from playlist.songs import looks_alike`; run pytest from the `playlist-lab` folder and they work as

written.

A5. Start your Claude chat with these rules

Open Claude in the `playlist-lab` folder (or paste code back and forth). **Paste this first, once, before Step 01:**

```
You are my coach for a small Python + pytest project in this folder (Python 3.9+).
Rules for the whole session:
1. We work in tiny steps. I will say which step. Do ONLY that step.
2. Test first. When I ask for a test, write the test (and a stub if needed), then STOP.
   Do not make a failing test pass until I ask you to.
3. Never edit tests/relations.py or playlist/track.py. Never change a test just to make
it pass.
4. Keep code short and plain. After the code, explain each new idea in one sentence of
everyday English.
5. All data is made up. Never use real people or real songs.
6. Tell me the exact command to run (python3 -m pytest -q) and what red or green I
should expect.
```

The prompts in the steps below are what you paste *after* that. Fill in the ____ blanks with **your** answers first.

Reading pytest output. Every run includes *all* the tests you have so far, so totals grow ("15 passed" at one step, "30 passed" later). That is normal. Look for the lines that say `FAILED` and the line starting with `E`, not for the total.

Open Claude in the folder: if you use Claude Code, start it from a terminal inside `playlist-lab`. If you use a chat window, paste the file's text in and copy Claude's file text out into your folder.

A6. Keep an honest log

When Claude gives you something, you will mark it in the log at the end: **AI proposed** it, and then **we checked it** by running it or working it out on paper. Both columns matter.

Part B. The steps

Step numbers match Tuesday's prompts one-for-one (00 to 14: same numbers, same jobs, different problem). A step with **a** and **b** is red, then green. Tuesday's demo ladder tags are listed in the facilitator page.

Class fast path (what we aim for together): Steps 01, 06, 07, 08. Step 01 shows a relation as pairs, matrix and arrows; Steps 06 to 08 are the heart: a rule that looks right, the counterexample *you* invent, and the repair. **At home or next class: Steps 02, 03, 04, 05, then 09 to 14,** the same moves again. Steps 06 to 08 need only the starter folder; they do not need Steps 02 to 05. Steps 09 to 14 need the files from Steps 06 to 08 (`playlist/songs.py` with `song_key`, and your Slow Orbit tracks in `tests/fixtures.py`).

BEAT 0. A relation is a list of pairs (Steps 01–03, warm-up)

These three steps are mostly **paper**. You predict, then Claude's script prints the answer so you can check yourself. They are the same moves as Tuesday's first three prompts, on a playlist.

Everyday picture. A playlist is a queue. "Track B plays right after track A" is a yes/no question about two tracks. A **relation** is just the list of all the *ordered pairs* (a, b) for which the answer is yes. An **ordered pair** is two things where the order matters: (Ana, Ben) is not the same as (Ben, Ana).

Our queue, in order: **Slow Orbit, Paper Moons, Glass Harbor, Night Bus.**

Step 01. Show a relation three ways: pairs, matrix, arrows

PREDICT (paper, before Claude). The relation is "plays right after".

List every ordered pair: _____ Fill the matrix: rows = first track, columns = second track, put 1 where the pair is in the relation, else 0.

	Slow Orbit	Paper Moons	Glass Harbor	Night Bus
Slow Orbit				
Paper Moons				
Glass Harbor				
Night Bus				

Draw four dots and an arrow for every pair. Is the matrix the same if you flip it over its diagonal? ____

ASK CLAUDE.

```
STEP 01. Create demos/step01_pairs.py (no tests yet). The queue is ["Slow Orbit", "Paper
Moons", "Glass Harbor", "Night Bus"].
The relation is "plays right after": (a, b) where b is the very next track after a.
Print three things:
(1) the ordered pairs one per line, (2) a 4x4 matrix of 0s and 1s with the track names
on the rows, (3) arrows like "A -> B".
Standard library only. Then run it with python3 demos/step01_pairs.py and show me the
output.
```

RUN. `python3 demos/step01_pairs.py` **You should see:** 3 pairs, a matrix with exactly three 1s that sit just above the diagonal, 3 arrows in a chain. Compare with your paper.

EXPLAIN. The matrix is *not* the same flipped over. In one sentence, what does that tell you about this relation?

___

Check: [ ] my pairs and matrix were written before the run [ ] mine and the computer's agree (or I know why not) **Stuck?** Which row has all zeros? That track plays last: nothing plays after it.

Step 02. Ask four yes/no questions, with a witness for every "no"

For any relation, ask four questions: **Reflexive?** is every track related to itself? **Symmetric?** if (a, b) is in, is (b, a)? **Antisymmetric?** if both (a, b) and (b, a) are in, must a and b be the same track? **Transitive?** if (a, b) and (b, c) are in, is (a, c)?

For every "no", we want a **witness**: the exact track or tracks that break it.

PREDICT (paper). For "plays right after":

> Reflexive: yes / no. If no, witness: _ *Symmetric: yes / no. If no, witness:* _ Antisymmetric: yes / no. If no, witness: _ *Transitive: yes / no. If no, witness:* _

ASK CLAUDE.

```
STEP 02. Create demos/step02_properties.py. Use reflexive_failure, symmetric_failure,
antisymmetric_failure and
transitive_failure from tests/relations.py (add tests/ to sys.path so the import works;
do not change that file; read it first and tell me in one sentence what
each one returns). For the "plays right after" relation on the queue from Step 01, print
one line per property:
"holds" or "FAILS" followed by the witness it returned. Run it and show me the output.
```

RUN. `python3 demos/step02_properties.py` **You should see:** three FAILS lines with witnesses and one "holds" (antisymmetric). If your paper disagrees, find the witness the computer found and test it by hand.

EXPLAIN. Antisymmetric "holds" even though the relation is not symmetric. Why is that not a contradiction?

Check: [ ] four predictions made before running [ ] I can read each witness aloud **Stuck?** Not sure what a finder returns? It returns the first counterexample as a tuple, or `None` if it found none.

Step 03. Name the kind of relation: equivalence, partial order, or neither

An **equivalence relation** is reflexive + symmetric + transitive: it sorts things into groups where members "count as the same." A **partial order** is reflexive + antisymmetric + transitive: "comes before or is equal to." If it is neither, say so.

PREDICT (paper). Decide for each, using the four questions. Write yes/no for the four properties, then a verdict.

Relation	Over	refl	sym	anti	trans	Verdict (you)
shares an artist with	Slow Orbit and Slow Orbit Remix (Lantern Hill), Paper Moons and Glass Harbor (Moss & Marrow)					
is no longer than	the lengths 180, 200, 240, 300					
title starts with the other's title	Slow Orbit, Slow Orbit Remix, Paper Moons, Glass Harbor					
is within 5 seconds of	the lengths 200, 204, 208					
plays right after	the queue from Step 01					

ASK CLAUDE.

```
STEP 03. Create demos/step03_classify.py. Define the five relations in the table I give
you below as small functions over
the lists I give you, run the four failure finders from tests/relations.py (add tests/
to sys.path) on each, and print a table with columns
refl, sym, anti, trans (yes/NO) and a verdict column: "equivalence" if refl+sym+trans,
"partial order" if refl+anti+trans,
otherwise "neither". Run it and show me the output. <paste the table above, without my
answers>
```

RUN. `python3 demos/step03_classify.py` **You should see:** verdicts, top to bottom: equivalence, partial order, partial order, neither, neither. Mark every cell where you were wrong; the wrong ones are the useful ones.

EXPLAIN. Pick one "neither". Which property failed, and what is its witness? ____

Check: [ ] all 20 cells predicted first [ ] I can say which of the five is NOT symmetric and why **Stuck?** "starts with": b starts with a means a is a *beginning* of b. Is that true in both directions for different titles?

BEAT 1. Cut things into bins (Steps 04–05)

Everyday picture. A post office charges by weight bracket: under 1 lb, 1 to under 5 lb, and so on. The only hard question is the *edge*: a parcel of exactly 1 lb, which bracket? Bugs live at the edges.

A **bin** (also called a **partition** cell) is a group of inputs the function treats the same way. The **boundary** is the pair of values on either side of the edge between two bins. Test one value from the middle of each bin, and test both sides of every boundary.

Step 04a (RED). Write the test for `length_label`

Goal. A function `length_label(seconds)` labels a track by length:

Label	Seconds
`"short"`	0 up to but not including 180
`"normal"`	180 up to but not including 300
`"long"`	300 up to but not including 600
`"epic"`	600 up to and including 3600
(error)	below 0 or above 3600: raise `ValueError`

PREDICT (before anything else). Fill in the test values you will use. For each bin, one value from the middle and the two values right at each edge.

> short: middle **_** ***edges*** *and* **_** ***normal: middle*** edges **_** ***and*** *long: middle* **_** ***edges*** and **_** ***epic: middle*** *edges* **_** ***and*** errors: **_** ***and*** *Which value do you think a lazy programmer would get wrong, and why?* **___**

Worked example for one bin (short = 0 up to but not including 180). Middle: 90. Edges: the *last* value in the bin (179) and the *first* value of the next bin (180). Do the same for each bin.

ASK CLAUDE. (Use your numbers; paste your filled-in table under the sentence about lengths.)

```
STEP 04a (red). Create playlist/lengths.py with a stub length_label(seconds) that just
returns None.
Create tests/test_lengths.py with plain assert tests, one assert per length value
(pytest.mark.parametrize is fine) so that a failure names the exact value, using exactly
these
lengths: <paste your table of values>. Add one test that -1 and 3601 raise ValueError.
Do NOT write the real function. Stop after the files exist.
```

RUN. `python3 -m pytest -q`

You should see: red. Several failures such as `assert None == 'short'`. That is good: the stub returns nothing, so every test notices.

EXPLAIN. Why is it good that these tests fail right now? **____**

Check: [ ] test file exists [ ] red result [ ] I can point to a test for each edge value **Stuck?** Red output looks like a wall of text? Read only the last line (`N failed`) and the line starting with `E` (it names the surprise). Test passes at Step 04a? Then the stub is not a stub; ask Claude to make it return `None` only.

Step 04b (GREEN). Make `length_label` pass

PREDICT. If the real function used `>` where it should use `>=` on the edge at 180, which of your test values would catch it? **__**

ASK CLAUDE.

```
STEP 04b (green). Now write the real length_label in playlist/lengths.py so all tests in
tests/test_lengths.py pass.
Smallest code that works. Do not touch the tests.
```

RUN. `python3 -m pytest -q` **You should see:** green (all passed).

EXPLAIN. In one sentence, what is each bin, in your own words? ____

Check: [ ] green [ ] I did not change any test **Stuck?** Still red? Look at the *first* failing test. Which of your edge values failed? That value tells you which > or >= is wrong. Ask Claude *why that one value* failed, not for new code.

Step 05 (GREEN from the start). "Same label" cuts the lengths into groups

A **relation** is a yes/no question about two things. Here: "do these two lengths get the same label?" A relation that is **reflexive** (each thing is related to itself), **symmetric** (if a relates to b then b relates to a) and **transitive** (a to b and b to c means a to c) is an **equivalence relation**: it sorts everything into groups with no overlap. "Same label" should be one. We will check, not hope.

This test is **green from the start** (it records something already true), which is called a **characterization test**.

PREDICT. Will it be green or red? _ *Why?* ___

ASK CLAUDE.

```
STEP 05. Create tests/test_length_relation.py. Define same_label(a, b) = length_label(a)
== length_label(b).
Using reflexive_failure, symmetric_failure and transitive_failure from
tests/relations.py, write three tests
that each assert the failure finder returns None, over the list [0, 90, 179, 180, 240,
299, 300, 450, 599, 600, 1800, 3600].
Add a fourth test that the labels cut that list into exactly four bins.
```

RUN. `python3 -m pytest -q` **You should see:** green. A finder returns `None` when it finds *no counterexample*. `None` is the good answer.

EXPLAIN. Pick one length from each bin. Name a pair that is "the same" and a pair that is not. ____

Check: [ ] green [ ] I know what `None` means from a finder **Stuck?** Import error? Run pytest from the `playlist-lab` folder, not a folder inside it.

BEAT 2. The rule that looks right but is not transitive (Steps 06–08)

Everyday picture. Chairs in a row, 4 feet apart. "Sitting near" means within 5 feet. Chair 1 is near chair 2. Chair 2 is near chair 3. But chair 1 is 8 feet from chair 3, so they are *not* near. "Near" does not chain. A rule that does not chain cannot sort people into tidy groups. This is the whole lesson of today.

Step 06a (RED). A first guess at "looks like the same song"

First attempt: two listings **look alike** if they have the same artist (ignore capital letters) and their lengths are within 5 seconds of each other.

PREDICT. Say whether each pair looks alike, then check with Claude's tests.

> artist Zed 100s vs artist Zed 104s: **_** ***artist Zed 100s vs 105s:*** *artist Zed 100s vs 106s:* **_** ***artist zed 100s vs ZED 101s:*** different artists, both 100s: ____

ASK CLAUDE.

```
STEP 06a (red). Create playlist/songs.py with a stub looks_alike(a, b) that returns
False.
Create tests/test_looks_alike.py with tests for: same artist 4 seconds apart; exactly 5
apart;
6 apart; capitals in the artist name differing; different artists with the same length.
Build the Tracks inline with Track(id, title, artist, seconds). Do NOT write the real
rule yet.
```

RUN / expect: red, but only *some* tests fail (the ones that expect `True`). The ones that expect `False` already pass because the stub always says `False`. That is normal.

EXPLAIN. Why do some tests pass even though the function does nothing? ____

Check: [ ] red with some passes [ ] my predictions written before running **Stuck?** Count a pair's gap by hand: `abs(100 - 106)` is 6. Over 5, so not alike.

Step 06b (GREEN). Make `looks_alike` real

ASK CLAUDE.

```
STEP 06b (green). Write the real looks_alike in playlist/songs.py: same artist ignoring
capitals, and lengths
within 5 seconds (5 apart still counts). Make tests/test_looks_alike.py pass. Do not
touch the tests.
```

RUN / expect: green.

EXPLAIN. Which of your PREDICT answers were wrong, and why? (If none, say what you were least sure about.) ____

Check: [ ] green [ ] I can state the rule in one sentence **Stuck?** One test still red? Is it the "exactly 5" test? Then the code needs <=, not <.

Step 07 (RED). Find the counterexample yourself

Now the key move. A **witness** is a concrete example that proves a claim. When the claim is "this rule always chains" and the example breaks it, it is a **counterexample**. Before the computer finds one, **you** find one.

`tests/fixtures.py` has ids 1, 2 and 3 saved for three listings of one song, "Slow Orbit" by "Lantern Hill". They are yours to write. The titles differ on purpose ("Slow Orbit", "Slow Orbit (Remastered)", "slow orbit"), and the third artist is written `LANTERN HILL`.

PREDICT. YOU invent the witness. Choose a length in seconds for each listing so that, under "within 5 seconds":

- listing 1 looks alike listing 2, **and**
- listing 2 looks alike listing 3, **but**
- listing 1 does **not** look alike listing 3.

> Listing 1: ***_ s Listing 2:*** *s Listing 3:* ***_ s Check by hand: gap 1→2 =*** (≤ 5?) gap 2→3 = ***_ (≤ 5?) gap 1→3 =*** _ (> 5?) Draw it: three dots labeled 1, 2, 3. Draw an arrow each way for every pair that looks alike. Which arrow is missing? ____

Do not ask Claude for these numbers. Add your three Tracks to `tests/fixtures.py`, following the pattern of `PAPER_MOONS`, and add them to the front of `TRACKS`:

```
SLOW_ORBIT_1 = Track(1, "Slow Orbit", "Lantern Hill", ____)
SLOW_ORBIT_2 = Track(2, "Slow Orbit (Remastered)", "Lantern Hill", ____)
SLOW_ORBIT_3 = Track(3, "slow orbit", "LANTERN HILL", ____)
TRACKS = [SLOW_ORBIT_1, SLOW_ORBIT_2, SLOW_ORBIT_3, PAPER_MOONS, GLASS_HARBOR,
NIGHT_BUS]
```

ASK CLAUDE.

```
STEP 07 (red). In playlist/songs.py add same_song(a, b) that for now just returns
looks_alike(a, b).
Create tests/test_transitivity.py using reflexive_failure, symmetric_failure and
transitive_failure from tests/relations.py
over TRACKS from tests/fixtures.py: three tests, each asserting the failure is None. For
the transitive test,
make the assert message print the ids and seconds of the three tracks it found.
Do not fix anything. Stop.
```

RUN / expect: one failure, the transitive test. The message names three ids and their seconds. Reflexive and symmetric pass.

EXPLAIN. Did pytest find the same three listings you did? Write the sentence: "Listing __ looks alike listing **, and listing __ looks alike listing** , but **____." Then say in plain words why a single rule that only compares two things at a time cannot be trusted to sort into groups. _____**

Check: [ ] I wrote my numbers *before* running [ ] red on transitivity only [ ] I can read my counterexample aloud **Stuck?** Cannot find numbers? **Wait ten minutes first.** Then use the chairs: each neighbouring pair may be at most 5 apart; how far apart can the two ends of a row of three be? All three tests pass? Then your numbers do not break chaining; check gap 1→3 is over 5.

Step 08 (GREEN). Make "same song" a real equivalence

The repair: stop asking "are these close?" and instead **normalize** each listing to a **key** (a standard form), and say two listings are the same song exactly when their keys are equal. Here the key is the artist and the title in lowercase, with anything in `( )` and any punctuation removed. Equal keys cannot fail to chain, because "equal" always chains.

Tradeoff, stated plainly: the key is stricter. It will not catch a typo in a title. We gave up some cleverness to get a guarantee.

PREDICT. What key do you expect for `"Slow Orbit (Remastered)"` by `"Lantern Hill"`? _ ***For `"slow orbit"` by `"LANTERN HILL"`?*** _ Equal? ____

ASK CLAUDE.

```
STEP 08 (green). In playlist/songs.py add song_key(track): artist lowercase and
stripped, title lowercase with any
(...) removed, punctuation removed, extra spaces collapsed; returns a tuple. Change
same_song to compare song_keys.
Keep looks_alike as it is. In tests/test_transitivity.py also add (a) a test that
looks_alike is STILL not
transitive over TRACKS (so our counterexample stays on record) and (b) a test that the
three Slow Orbit tracks
have the same song_key, that song_key(SLOW_ORBIT_2) == ("lantern hill", "slow orbit")
exactly, and that Paper Moons does not.
Punctuation is removed from the title only; the artist is just lowercased and stripped.
```

RUN / expect: green.

EXPLAIN. In your own words, why is "equal keys" safe where "close lengths" was not? One sentence: ____

Check: [ ] green [ ] the old counterexample is still a test [ ] I can state the tradeoff **Stuck?** `song_key` gives the wrong key? Ask Claude to print `song_key` for your three tracks, and compare with your PREDICT line.

Core done (through Step 08)? If so, jump to Part C and write your summary. Then keep going.

BEAT 3. Group by chains (Step 09)

Everyday picture. "Friend of a friend" is in the group: if Ana knows Ben and Ben knows Cy, all three end up in one friend group even if Ana and Cy never met. Chains of matches build groups.

Step 09a (RED). Cluster tracks by chains of `looks_alike`

PREDICT. Using `TRACKS` and `looks_alike`, how many groups, and who is in each? (Use ids.) ____

ASK CLAUDE.

```
STEP 09a (red). Create playlist/grouping.py with a stub cluster_by_chain(items, rel)
that returns each item in its own list.
Create tests/test_grouping.py over TRACKS with looks_alike: the three Slow Orbit tracks
share one cluster;
Paper Moons, Glass Harbor and Night Bus are each alone; every track is in exactly one
cluster.
Do NOT write the real function.
```

RUN / expect: red (the Slow Orbit cluster test fails).

EXPLAIN. Why must it be exactly one cluster per track, never two? ____

Check: [ ] red [ ] my group prediction written first

Step 09b (GREEN). Make clustering real

ASK CLAUDE.

```
STEP 09b (green). Write the real cluster_by_chain in playlist/grouping.py using union-
find (connected components):
if rel(a, b) is True put them in the same cluster, and chains join clusters. Return a
list of lists.
Make tests/test_grouping.py pass. Explain union-find in two sentences of everyday
English.
```

RUN / expect: green.

EXPLAIN.

(1) This clustering used `looks_alike`, the rule that failed to chain. Why does clustering by chains still give clean groups? ___ *Hint: chaining means a match joins the group even if the ends are far apart. (2) Tradeoff, think and write (drift): chains of small steps can keep growing a group, so songs 50 seconds apart could end up together. If Lantern Hill also had a different song of length 206 seconds, which cluster would it land in? Is that what we want?* ___ **Stuck?** Re-read the friend-of-a-friend picture.

BEAT 4. Is "duplicate" the same both ways? (Step 10)

Everyday picture. "Is the sibling of" works both ways. "Is the boss of" does not. A relation that works both ways is **symmetric**. "Duplicate" should be symmetric: if A is a duplicate of B, B is a duplicate of A. But careless wording ("B is a repeat of A, because B came later") is directional.

Step 10a (RED). `is_duplicate` must be symmetric

PREDICT. If the rule is "same song, and B was added after (or at the same time as) A", is `is_duplicate(1, 2)` true? And `is_duplicate(2, 1)`? _ _ Symmetric?: ____

ASK CLAUDE.

```
STEP 10a (red). Create playlist/duplicates.py with is_duplicate(a, b) = same_song(a, b)
and b.id >= a.id.
Create tests/test_duplicates.py using reflexive_failure and symmetric_failure over
TRACKS, each asserting None;
the symmetric message must print both ids and say which direction is True. Do not fix
the rule.
```

RUN / expect: red on symmetric only; the message shows `is_duplicate(1, 2)` True but `is_duplicate(2, 1)` False.

EXPLAIN. Write the counterexample in one sentence in your own words. ____

Step 10b (GREEN). Make it symmetric

ASK CLAUDE.

```
STEP 10b (green). Fix is_duplicate so swapping a and b can never change the answer.
Which copy to KEEP is a separate
question that we handle later. Make tests/test_duplicates.py pass.
```

RUN / expect: green. **EXPLAIN.** Why is "which one do I keep?" a different question from "are these duplicates?" ____

BEAT 5. Ties and order (Step 11)

Everyday picture. A photo-finish: two runners have exactly the same time. "A finished at or before B" is true *both ways* for two different runners. A proper order, **antisymmetric** (if A is at-or-before B *and* B is at-or-before A, they must be the same thing), needs a tiebreaker such as bib number. An order that is reflexive, antisymmetric and transitive is a **partial order**.

Step 11a (RED). Ties break the order

PREDICT. Which two tracks in `TRACKS` have exactly the same length, so a length-only ordering cannot tell them apart? ids _ ***and*** _

ASK CLAUDE.

```
STEP 11a (red). Create playlist/ordering.py with compare(a, b) returning -1, 0 or 1 by
comparing seconds only.
Create tests/test_ordering.py: define at_or_before(a, b) = compare(a, b) <= 0; test
antisymmetric_failure is None
(message names the two different tracks), transitive_failure is None, and that sorting
every shuffle of TRACKS
(itertools.permutations, functools.cmp_to_key; for 6 tracks that is 720 orderings) gives
exactly one answer. Do not fix it.
```

RUN / expect: red on antisymmetric and on "one answer". Did the two ids match your prediction?

EXPLAIN. Why does sorting give different results depending on how the list started? ____

Step 11b (GREEN). Add a tiebreaker

ASK CLAUDE.

```
STEP 11b (green). Make compare use a unique sort key: (seconds, artist lowercase, title
lowercase, id).
compare may return 0 only for the very same track. Make tests/test_ordering.py pass.
```

RUN / expect: green. **EXPLAIN.** Why does adding `id` fix it? ____

BEAT 6. Put the pieces together (Step 12)

Everyday picture. After you have tested the screws and the planks, you can build the shelf.

Step 12a (RED). `clean_playlist`

Rule: keep one track per song, the earliest added (smallest id), and return them in id order.

PREDICT. What does `clean_playlist(TRACKS)` return? List the ids. __ And if you run it on its own result? __

ASK CLAUDE.

```
STEP 12a (red). Create playlist/cleaner.py with a stub clean_playlist(tracks) that
returns list(tracks).
Create tests/test_cleaner.py: TRACKS gives the earliest listing of each song in id
order; a playlist with no
duplicates comes back unchanged; an empty playlist stays empty; the reversed input
(reversed(TRACKS)) gives the same result; cleaning
twice gives the same answer as cleaning once. Do NOT write the real function.
```

RUN / expect: red (the duplicate and shuffle tests fail; the "no duplicates", "empty" and "twice" tests already pass with the stub). **EXPLAIN.** Which tests passed with the stub, and what does that tell you about them? ____

Step 12b (GREEN). Make it real

ASK CLAUDE.

```
STEP 12b (green). Write the real clean_playlist using song_key to group the tracks and
keep the earliest id of each group.
Make tests/test_cleaner.py pass.
```

RUN / expect: green. **EXPLAIN.** Which earlier step made this function easy to write? ____

BEAT 7. What depends on what? (Steps 13–14)

Everyday picture. Socks before shoes. Shirt before jacket. Some things must come first; some do not care about each other. And a **cycle** is a trap: "the key is in the room, and I need the key to enter the room." A "needs" order has no cycles. Test the pieces nothing else needs *first*; the idea is "leaves first".

Do this on paper first. Here are our functions and what each one *uses*: `clean_playlist` uses `song_key`; `is_duplicate` uses `same_song`; `same_song` uses `song_key`; `song_key`, `looks_alike`, `cluster_by_chain`, `compare` and `length_label` use no other function of ours. Draw an arrow from each function to what it needs. **Which functions would you test first? __**

Step 13a (RED). Order the tests leaves-first

ASK CLAUDE.

```
STEP 13a (red). Create playlist/depends.py with a CycleError exception and a stub
leaves_first(graph) returning list(graph).
Create tests/test_depends.py with this graph written dependents first:
{"clean_playlist": {"song_key"},
"is_duplicate": {"same_song"}, "same_song": {"song_key"}, "song_key": set(),
"looks_alike": set(),
"cluster_by_chain": set(), "compare": set(), "length_label": set()} where "X": {"Y"}
means "X needs Y".
Tests: every function comes after the ones it needs; every function appears exactly
once. Do not write the real one.
```

RUN / expect: red. **EXPLAIN.** (Paper answer to check later: the functions that use nothing else, `song_key`, `looks_alike`, `cluster_by_chain`, `compare`, `length_label`, come first; `same_song` next; `is_duplicate` and `clean_playlist` last.) Why does test order matter for finding bugs? ____

Step 13b (GREEN). Make it real

ASK CLAUDE.

```
STEP 13b (green). Write leaves_first as a depth-first walk: visit what a function needs
first, then add the function.
Mark each function as seen so it is added once. Make tests/test_depends.py pass.
```

RUN / expect: green. **EXPLAIN.** Compare Claude's order with your paper answer. ____

Step 14a (RED). A cycle should be an error

PREDICT. For `{"a": {"b"}, "b": {"a"}}` what should the order be? Is there a right answer? _____

ASK CLAUDE.

```
STEP 14a (red). Create tests/test_cycles.py: leaves_first({"a": {"b"}, "b": {"a"}})
raises CycleError; and for
{"a": {"b"}, "b": {"c"}, "c": {"a"}, "d": set()} the message contains "a -> b -> c ->
a". Do not fix it.
```

RUN / expect: red (no error raised yet).

Step 14b (GREEN). Detect the cycle

ASK CLAUDE.

```
STEP 14b (green). Make leaves_first raise CycleError, naming the loop, when functions
need each other. Track the current path while walking. Make all tests pass.
```

RUN / expect: green, all tests. **EXPLAIN.** What does a cycle in a "needs" order tell you about the design? ____

Part C. Your summary and evidence

C1. Your One-Sentence Summary (write this yourself, no Claude)

Use this pattern, in your own words:

> "I showed that __ **is / is not an equivalence relation, because** _ ***(the counterexample:*** ___), and I fixed it by _____."

C2. The honesty log

Step	What Claude proposed (AI proposed)	How we checked it (ran it / worked by hand)	Did the check agree?

C3. Evidence sheet for your pair (one page, both names)

- Last step we finished: ____ (core ends at 08)
- Paste or photograph the **red** output at Step 07 and the **green** output at Step 08.
- Our counterexample in words: ____
- One prediction we got wrong, and why: ____
- One thing we would change about our rule next time: ____

C4. Before you leave: the checklist

- [ ] I predicted before every run.
- [ ] I can say what red means and why it can be good.
- [ ] I wrote the counterexample myself.
- [ ] I can say why "close" is not an equivalence and "equal key" is.
- [ ] I marked which things Claude proposed and which we verified.
- [ ] I wrote my own One-Sentence Summary.